

# 2023114009

## Class Activity

1. One INSERT on EMPLOYEE that is acceptable

```
INSERT INTO EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)
VALUES ('Shiv', 'P', 'Singh', 555555555, '1971-05-05', '123 Oak St, Houston, TX', 'M', 45000, 333445555, 5);
```

2. One INSERT that violates key constraint

```
INSERT INTO EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)
VALUES ('John', 'B', 'Doe', 123456789, '1965-01-09', '731 Fondren, Houston, TX', 'M', 30000, 333445555, 5);
```

This violates key constraints because John Smith already has that `Ssn`

3. One INSERT that violates referential integrity constraints

```
INSERT INTO EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)
VALUES ('Yefri', 'M', 'Reyes', 987654321, '1992-03-21', '456 Elm St, Dallas, TX', 'F', 48000, 333445555, 10);
```

This violates referential integrity constraints because `Dno` is 10 but no department exists in the table with `Dnumber = 10`

4. One DELETE on EMPLOYEE that is acceptable

```
DELETE FROM EMPLOYEE WHERE Ssn = 555555555;
```

Valid because this `Ssn` is not referenced anywhere else, as we just added this entry in (1) so referential integrity remains intact.

*Note:* if we are supposed to perform these actions by assuming the given state of the database (and not the current state, after performing the above actions), **then no acceptable DELETE is**

possible, as the `WORKS_ON` table references all employees' `Ssn`s as foreign keys in the `Essn` column

5. One DELETE on EMPLOYEE that violates referential integrity constraints

```
DELETE FROM EMPLOYEE WHERE Ssn = 333445555;
```

Violates because this `Ssn` is referenced several times in the database, like in the `Super_ssn` of John Smith.

6. One UPDATE on EMPLOYEE that is acceptable

```
UPDATE EMPLOYEE SET Salary = 38000 WHERE Ssn = 453453453;
```

7. One UPDATE on EMPLOYEE that violates referential integrity constraints

```
UPDATE EMPLOYEE SET Dno = 7 WHERE Ssn = 333445555;
```

Because no department with `Dnumber` = 7 exists

8. One UPDATE on EMPLOYEE that violates primary key constraint

```
UPDATE EMPLOYEE SET Ssn = 123456789 WHERE Ssn = 333445555;
```

Because John Smith already has that `Ssn`

9. One INSERT on DEPARTMENT that is acceptable

```
INSERT INTO DEPARTMENT (Dname, Dnumber, Mgr_ssn, Mgr_start_date)
VALUES ('Marketing', 6, 888665555, '1970-01-01');
```

10. One INSERT on DEPARTMENT that is NOT acceptable

```
INSERT INTO DEPARTMENT (Dname, Dnumber, Mgr_ssn, Mgr_start_date)
VALUES ('IT', 5, 999999999, '2023-06-01');
```

Because `Dnumber` is a primary key and Research already has `Dnumber` = 5

11. One DELETE on PROJECT that is acceptable

```
DELETE FROM PROJECT WHERE Pnumber = 50; /* doesnt delete anything */
```

This statement does not delete any row because no row exists with `Pnumber = 50`

*Note:* If we must perform a DELETE, and aren't allowed to use an acceptable DELETE statement that doesn't actually delete a row, **then no acceptable DELETE is possible on PROJECT**, as all rows have `Pnumber` that is referenced in the WORKS\_ON table. Deleting any row would violate referential integrity

#### 12. One DELETE on PROJECT that is NOT acceptable

```
DELETE FROM PROJECT WHERE Pnumber = 1;
```

Violates referential integrity for the reason mentioned above

#### 13. One UPDATE on WORKS\_ON that is acceptable

```
UPDATE WORKS_ON SET Hours = 25.0 WHERE Essn = 123456789 AND Pno = 1;
```

Only 1 row has `Essn = 123456789` and `Pno = 1`, so only one UPDATE is performed

#### 14. One UPDATE on WORKS\_ON that is NOT acceptable

```
UPDATE WORKS_ON SET Pno = 50 WHERE Essn = 123456789 AND Pno = 1;
```

This violates referential integrity as `Pno` references `Pnumber` from PROJECT, and PROJECT does not have any entry with `Pnumber = 50`